段立波　32岁

邮箱：380469930@qq.com　电话：17645633032　性　别：男　工作年限：9年

技能清单

- 熟练掌握HTML、Javascript、CSS、函数式编程和模块化、微内核、插件化思想
- 浏览器渲染原理、页面性能优化、用户体验、前端监控、有过深入了解及实践
- 熟悉 React、Vue 相关技术栈及技术框架实现原理，有基于 React 的低（无）代码平台经验
- 丰富的前端项目架构经验，掌握Webpack，Vite 等构建工具，从0到1搭建复杂应用，掌握monorepo架构，使用cz-git、commitlint、husky、lint-stage等工具链，在开发规范、代码校验、测试、打包构建、发布、监控等整个上线环节有深入理解和实践
- 推进并主导团队基建落地，如：业务组件库基础库、脚手架等，提升团队协同开发效率
- 熟悉 Node]S 服务端开发，有 SSR 相关实践
- 熟悉CI、CD自动化部署流程，在项目中实施自动化构建与部署
- 了解AI相关基本概念，AI应用开发流程、使用LangChain开发应用
- 团队管理经验，协调团队成员，喜欢分享，通过推动团队内部技术分享解决技能不平衡和协作问题

工作经历

2024/12 – 2025/11　杭州氪极度引力科技有限公司（迅雷子公司）　前端开发负责人

- 负责团队从0到1搭建、技术选型决策，统筹移动端 / Web 端整体架构，建立标准化开发流程（含代码校验、自动化测试、发布），效率提升 30%；
- 主导IM、支付等核心模块开发，管控代码质量与项目进度，优化支付模块交互逻辑，用户支付流失率降低 15%

2022/05 – 2024/09　杭州浙安互娱网络科技有限责任公司　前端开发负责人

- 牵头 PC 端项目重构，解决构建慢、单独部署痛点，重构后 PC 端构建时间缩短 40%，取消单独部署流程；
- 定制化 WebSocket 实时通讯方案，用插件化思想开发复杂计算模块，解决大数据量渲染卡顿问题。优化 WebSocket 重连机制与数据分片渲染，实时行情页面卡顿率从 25% 降至 3%；插件化模块复用率达 70%，新功能开发周期缩短 50%
- 负责移动端优化、游戏内活动模块开发、CI/CD 自动化落地，设计游戏活动可视化配置工具，运营自主配置效率提升 60%，减少前端重复开发量。

2020/07 – 2022/03　微拍堂文化创意有限公司　产品活动运营业务线前端负责人

- 设计低代码平台技术架构，推进前端工程化全链路迭代，用技术提升业务价值
- 低代码平台支持 80% 活动页面零代码生成，前端人力投入减少 60%；工程化落地后线上 bug 率下降 45%，版本发布周期从 7 天压缩至 3 天
- 推动并主导组件库设计与开发，统一前端开发规范

2019/07 –　2020/07　天行云供应链有限公司　高级前端开发

- 开发中台项目与移动端商城，搭建前端性能监控体系，完善日志、监控、告警机制。
- 监控体系覆盖 95% 前端异常场景，问题定位时间从 2 小时缩短至 15 分钟；移动端商城首屏加载时间优化 35%，用户留存率提升 10%。

2016/06 – 2019/07　巴卡斯科技有限公司杭州分公司　前端开发

- 推进项目从JQuery迁移至React单页应用，Node.js开发公众号与中间层，搭建数据可视化平台。
- React迁移后页面交互响应速度提升50%；数据可视化平台支持10+业务报表实时生成，为决策提供精准数据支撑。

项目经验

低代码平台

业务介绍：主导低代码平台的核心架构设计与性能优化工作，涵盖编排引擎、逻辑配置、实时预览等实现平台高可用性和扩展性，解决原来开发大型活动需20+后端 ，10+前端投入，开发周期长，人力成本高、迭代慢问题。

技术栈：React、TypeScript、pnpm、Turborepo、InteractJs、Vite、monorepo

工作内容和成果：

·

试用水印

【架构设计】采用 monorepo 架构，整合编排引擎、变量管理器、逻辑配置模块，实现插件化扩展（支持自定义组件 / 流程）；

【工程化】推进团队规范工程化，包含 eslint9、stylelint、commitlint 相关规范约定与落地，极大提升新人上手速度和整体维护成本

【核心功能】设计 DSL 描述页面结构，用 interact.js 实现高级拖拽，iframe 沙箱 + postMessage 跨窗通信实现实时预览；

【性能优化】编辑器与预览器解耦，针对项目打包构建与产物进行专项优化，包含 split chunk、cdn、prefetch 等处理，首屏加载时间降低 30%。

【数据成果】开发周期从6周缩短至1周，前端人力投入减少80%；平台支持30+活动复用。

前端监控 SDK 及 SaaS 平台架构设计

业务介绍：主导并构建企业级前端监控 SDK与 SaaS 平台，实现监控采集、数据处理和可视化展示，形成覆盖前端性能和异常监控的完整解决方案。

技术栈：TypeScript、PerformanceObserver API、window.onerror、window.onunhandledrejection、Nest.js、Kafka、Clickhouse、React、shadcn/ui

工作内容和成果：

【架构】设计并实现了基于核心包和适配包的 SDK 架构，通过插件化机制支持多框架(React、Vue、VanilaJs)集成，支持自定义监控项和扩展性增强

【效率】深入优化 TypeScript 项目结构和配置，提高了静态检查的精准度和代码安全性，实现了高效的模块化管理。

【思考】构建了全面的性能和异常监控方案，覆盖了 FCP、LCP、CLS等多项核心指标和 JavaScript异常采集，提升了监控的广度与深度。

【优化】使用 Nest.is 和 Kafka 设计并构建了数据采集和处理的后端系统，实现了高并发数据处理，系统的响应性能提升了 50%。

【可视化】基于 React 和 shadcn/ui 搭建 SaaS 平台，实现了数据的多维度可视化展示和实时监控，为业务决策提供了可视化支持。

试用水印

微前端

业务介绍：本项目针对管理后台“巨石应用”编译慢、首屏卡顿等核心痛点，以Qiankun微前端架构为核心进行技术升级。通过按业务域拆分独立子应用实现模块化开发部署

技术栈: TypeScript、React、Qiankun、Vite

工作内容和成果：

【架构设计】采用 Qiankun 微前端架构重构“巨石应用”，按业务域拆分订单、用户、活动独立子应用，实现子应用模块化开发与独立部署，通过应用注册机制与沙箱隔离保障多应用协同稳定性。

【核心功能】构建子应用生命周期管理体系，支持子应用加载/卸载/更新的自动化调度；配置平台内置环境校验与配置模板，适配海外多区域部署差异；实现子应用与主应用的通信桥接，保障跨应用数据交互流畅。

【性能优化】基于 Lighthouse/PageSpeed 性能指标建立优化体系，通过 Vite 提升编译效率，结合资源懒加载（路由/组件级）、CDN 加速静态资源、HTTP 缓存策略优化，全方位降低首屏加载耗时。

【数据成果】管理后台编译速度提升 50%；首屏渲染时间从 3s 降至 1s 内；

全球站核心业务+组件库、脚手架基建

业务介绍：（1）解决实时行情大盘计算量大、Websocket 数据波动频繁，页面交互卡顿，数据可靠性低业务问题。（2）推进组件库落地与方案设计。（3）脚手架：涵盖项目初始化、依赖管理、开发流程管理、代码质量控制、自动化构建与部署等核心功能

技术栈: TypeScript、React、Css-in-js、Vite、pnpm-workspace、tsup、commander、Jest、Husky

工作内容和成果：

【业务侧优化】用 WebWorker 剥离复杂数据计算，避免阻塞主线程、基于Math.js封装链式调用解决复杂计算问题、结合WebSocket 断开重连 + 数据缓存恢复机制，保证数据准确性；

【脚手架】开发和优化命令行工具的核心功能，基于插件化架构设计命令行工具，推进开发和部署流程的自动化，利用 tsup 和 turbo 提高项目构建和部署速度，结合 husky 和commitlint进行代码提交检查，实现了高度自动化的工作流，构建和部署时间减少了 40%，显著提高了开发效率，主导多种框架的项目模板开发，支持 React、Vue 和 Vanilla JS 等框架，使用 giget 动态管理模板，快速响应项目初始化需求

【组件库】 monorepo架构搭建，eslint+stylelint+commitlint保障质量，tsup打包、repress生成文档、Jest+React Testing Library进行单元测试、Semntic Release自动化版本控制和发布。

社交项目

业务介绍：社交产品初期迭代慢，纯 React 架构无法满足 SEO 需求；PC 与 H5 端需单独开发，维护成本高；页面无限滚动时卡顿，首屏加载耗时久，影响用户留存。

技术栈: TypeScript、React、TailwindCSS、Vite、Next

工作内容和成果：

【架构重构】决策采用 Next.js 替代纯 React，实现 SSR（服务端渲染），兼顾快速迭代与 SEO 优化，提升搜索引擎收录率；

【跨端适配】设计响应式组件方案，基于 TailwindCSS 统一样式体系，复用一套代码库支持 PC 与 H5 端，减少 80% 跨端重复开发工作；

【第三方集成】整合腾讯 IM 实现即时通讯功能，对接 Stripe、Apple Pay、Google Pay 等多渠道支付，满足全球用户支付需求；

【性能优化】采用虚拟滚动列表（react-window）减少 DOM 节点数量，结合图片懒加载 + 占位符技术，优化无限滚动体验；对首屏资源进行预加载与代码分割，降低加载体积。

【数据成果】核心操作页面平均 FPS 从 25 提升至 52，卡顿现象基本消除；首屏平均渲染时间从 2s 降至 0.5s，用户首次交互等待时长缩短 75%；跨端维护成本降低 60%，产品迭代周期从 2 周压缩至 1 周，用户留存率提升 18%。

教育背景

2012.09-2016.06	黑龙江工程学院	数学系-信息与计算科学	本科	党员

个人优势

具备扎实的前端技术功底与工程化落地能力，同时拥有强逻辑思维与跨团队沟通协作意识，注重用户体验细节与代码质量，以结果为导向，兼具业务洞察力与持续学习的自驱力，能高效应对复杂需求与技术迭代，为团队提效与业务落地赋能。

试用水印